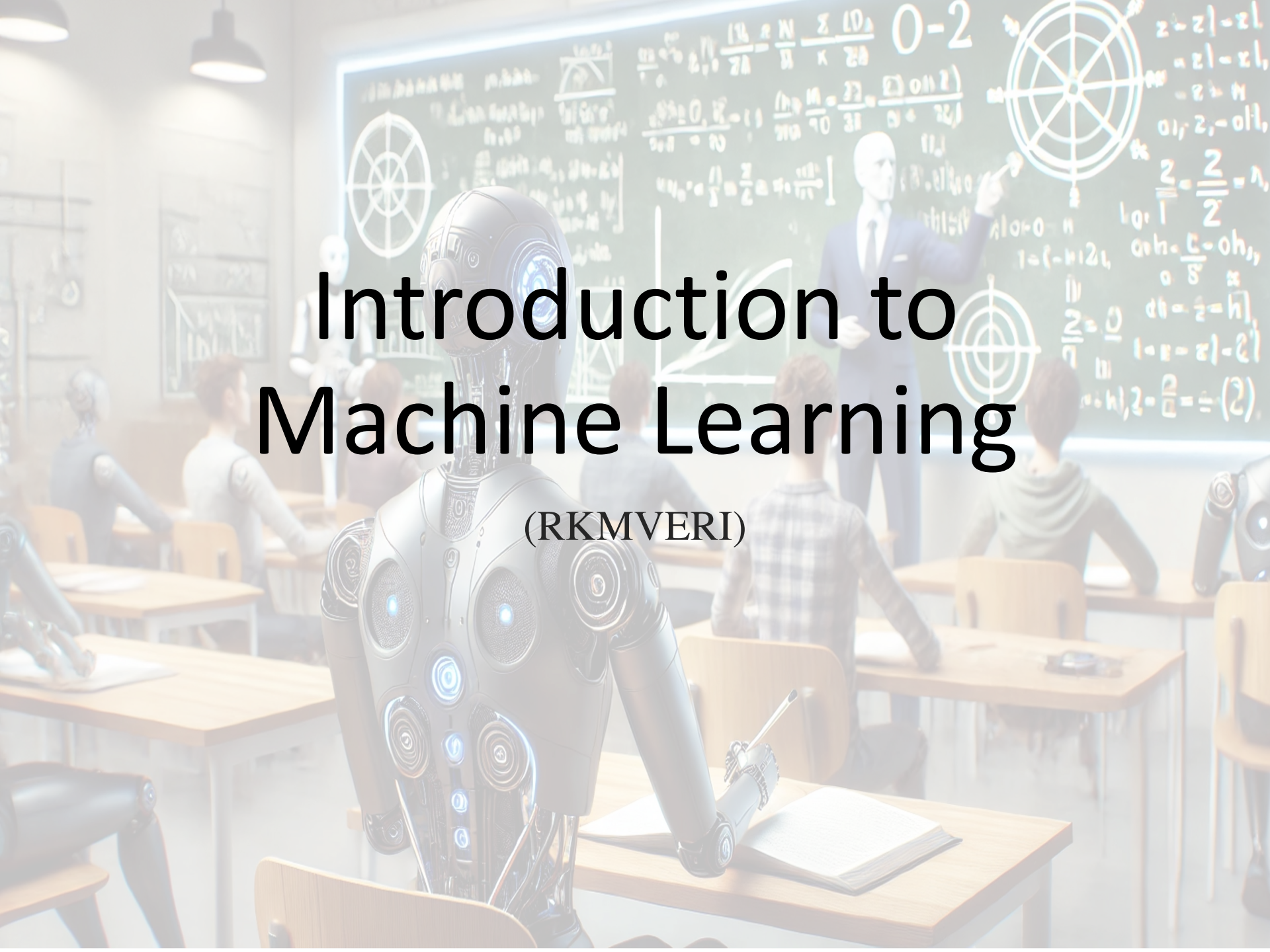




The background image depicts a classroom setting. In the foreground, a robot with a metallic, segmented body and glowing blue circular lights on its chest and joints is seated at a wooden desk, holding a pen over an open book. In the background, a teacher robot in a blue suit stands near a large chalkboard, pointing at it. The chalkboard is filled with various mathematical formulas, including the quadratic formula
$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$
, the area of a circle
$$A = \pi r^2$$
, and the volume of a sphere
$$V = \frac{4}{3}\pi r^3$$
. There are also diagrams of a target and a clock face. The overall scene is dimly lit, with a warm, yellowish glow from the classroom lights.

Introduction to Machine Learning

(RKMVERI)

Shanti Mantra

ॐ सह नाववतु ।

सह नौ भुनक्तु ।

सह वीर्यं करवावहै ।

तेजस्वि नावधीतमस्तु मा विद्विषावहै ।

ॐ शान्तिः शान्तिः शान्तिः ॥

Om Saha Nau-Avatu |

Saha Nau Bhunaktu |

Saha Viiryam Karavaavahai |

Tejasvi Nau-Adhiitam-Astu Maa Vidvissaavahai |

Om Shaantih Shaantih Shaantih ||

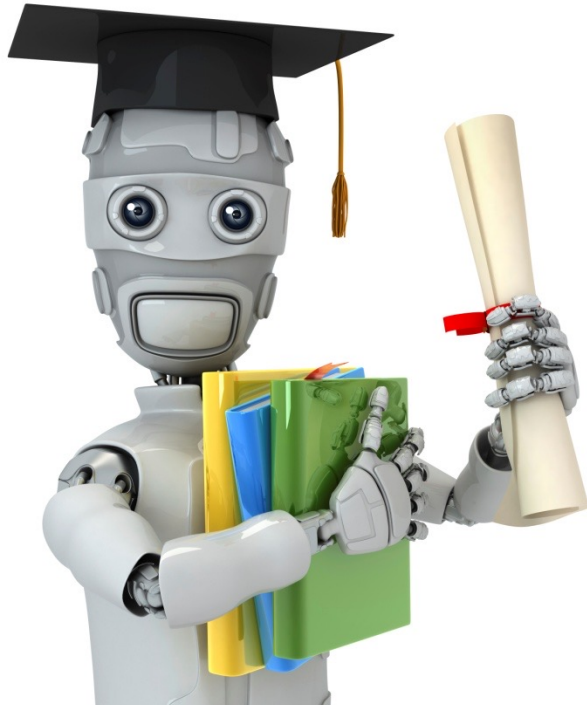
Om, may He protect us both (the teacher and the student).

May He nourish us both.

May we work together with energy and vigor.

May our study be enlightening and not give rise to hostility.

Om, peace, peace, peace.



Machine Learning

Logistic Regression

Multi-class classification

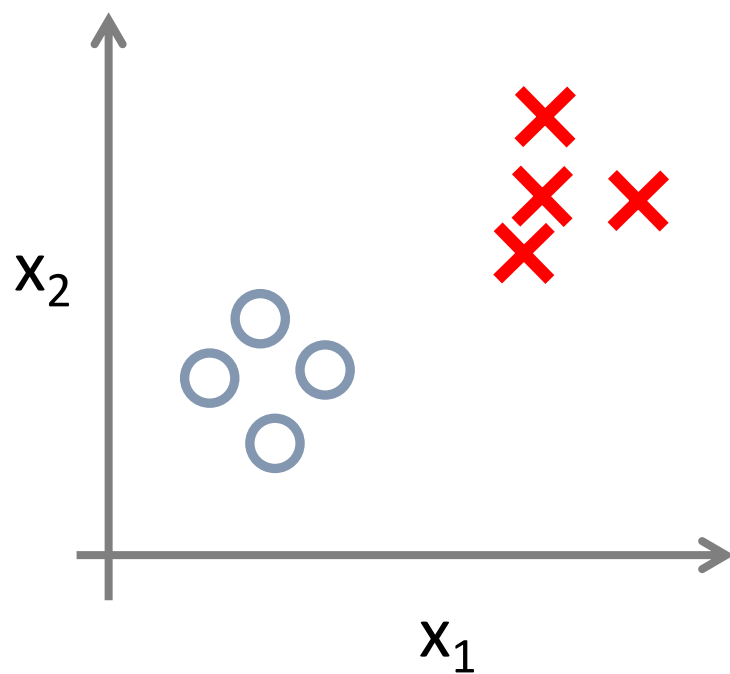
Multiclass classification

Email foldering/tagging: Work, Friends, Family, Hobby

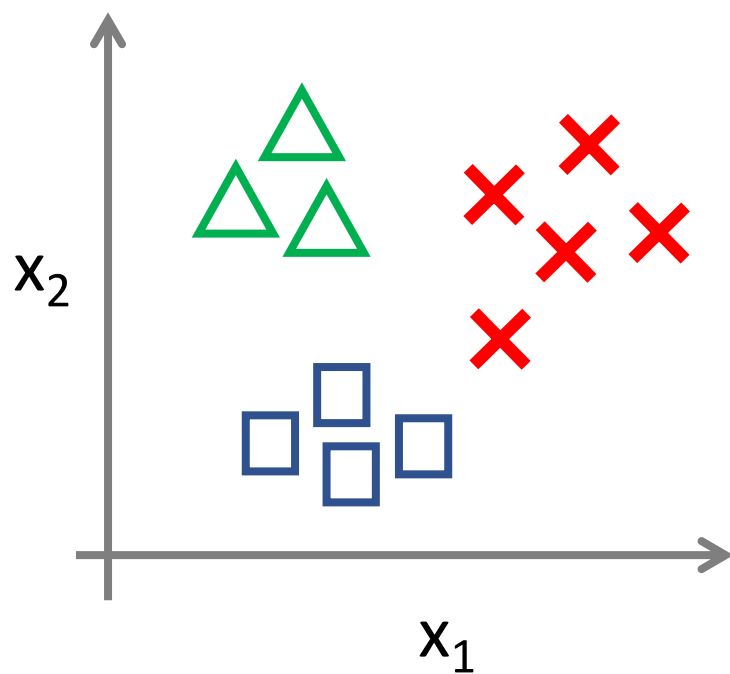
Medical diagrams: Not ill, Cold, Flu

Weather: Sunny, Cloudy, Rain, Snow

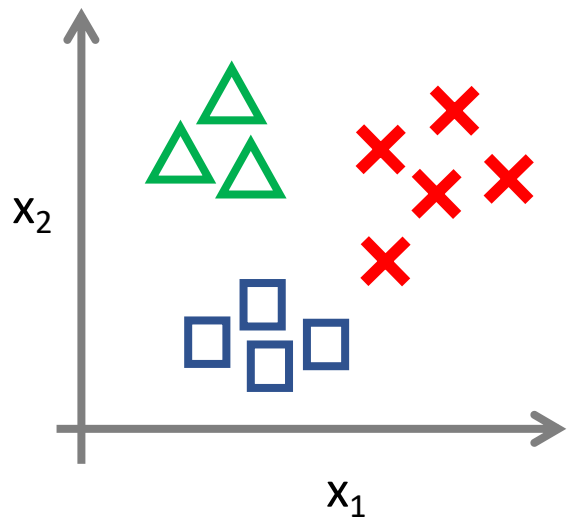
Binary classification:



Multi-class classification:



One-vs-Rest (OvR) / One-vs-All (OvA)

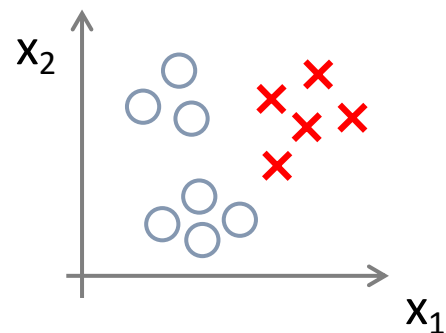
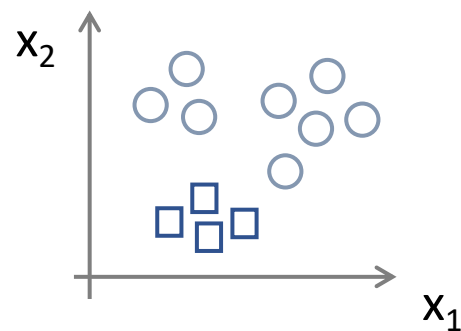
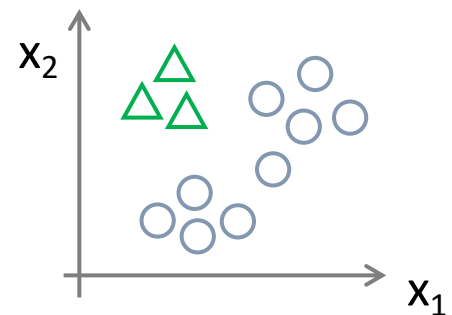


Class 1: 

Class 2: 

Class 3: 

$$h_{\theta}^{(i)}(x) = P(y = i|x; \theta) \quad (i = 1, 2, 3)$$



Various Multi-class classification

- One-vs-Rest (OvR) / One-vs-All (OvA)
- One-vs-One (OvO)
- Softmax Regression

One-vs-Rest (OvR) / One-vs-All (OvA)

For a classification problem with K distinct classes:

Train K binary classifiers, where the k -th classifier targets class k vs. all other classes.

For a new input $\mathbf{x}$, predict the class k that maximizes the probability:

$$\text{Predicted class} = \arg \max_k P(y = k \mid \mathbf{x}) = \arg \max_k h_{\theta}^{(k)}(\mathbf{x})$$

One-vs-Rest (OvR) / One-vs-All (OvA)

- **Advantages:**

- **Simplicity:** OvR is easy to implement and understand, making it a popular choice in many applications.
- **Scalability:** Works well with a moderate number of classes, and each binary classifier can be trained independently, allowing for parallel processing.

- **Disadvantages:**

- **Efficiency:** Training K separate models can be computationally expensive, especially when K is large.
- **Probability Interpretation:** The probabilities produced by each classifier are independent, which can lead to inconsistencies, as the sum of probabilities across all classes may not equal 1.

One-vs-One (OvO)

- A separate binary classifier is trained for each pair of classes, effectively breaking down the multiclass problem into multiple two-class problems.

For K classes, $\frac{K(K-1)}{2}$ binary classifiers are trained, one for each pair of classes

- During prediction, each binary classifier votes for one of the two classes it was trained on.
- The final class label is determined by majority voting, where the class receiving the most votes across all classifiers is chosen as the prediction.

$$\text{Predicted class} = \arg \max_k \text{votes}_k$$

For the MNIST problem, this means training 45 binary classifiers!

One-vs-One (OvO)

Advantages:

- **Efficiency:** Can be more computationally efficient than OvR when the number of classes is large because each classifier deals with only two classes at a time, reducing the complexity of each individual model.
- **Handling Class Overlap:** Performs better where there is significant overlap between classes, as each classifier focuses on a binary distinction.

Disadvantages:

- **Number of Classifiers:** The number of classifiers grows quadratically with the number of classes $\left(\frac{K(K-1)}{2}\right)$, which can become computationally expensive and complex to manage as K increases.
- **Voting Ties:** In some cases, voting process can result in ties, requiring additional tie-breaking logic.

Softmax Regression

- The logistic regression model can be generalized to support multiple classes directly, without having to train and combine multiple binary classifiers
- This is called softmax regression, or multinomial logistic regression.
- This treats the problem as a single optimization problem with a unified probabilistic framework.

Softmax Regression

Given an input instance $\mathbf{x}$, the softmax regression model computes a score $s_k(\mathbf{x})$ for each class k .

$$s_k(\mathbf{x}) = \left(\boldsymbol{\theta}^{(k)} \right)^T \mathbf{x}$$

The equation used to compute $s_k(\mathbf{x})$ resembles the equation for linear regression prediction.

The model then estimates the probability of each class by applying the **softmax function** (also known as the **normalized exponential**) to these scores.

Softmax function:

$$\hat{p}_k = \sigma(\mathbf{s}(\mathbf{x}))_k = \frac{\exp(s_k(\mathbf{x}))}{\sum_{j=1}^K \exp(s_j(\mathbf{x}))}$$

Each class has its own dedicated parameter vector $\boldsymbol{\theta}^{(k)}$, and these vectors are typically stored as rows in a parameter matrix $\boldsymbol{\Theta}$.

Softmax Function

$$\hat{p}_k = \sigma(\mathbf{s}(\mathbf{x}))_k = \frac{\exp(s_k(\mathbf{x}))}{\sum_{j=1}^K \exp(s_j(\mathbf{x}))}$$

K is the number of classes.

$\mathbf{s}(\mathbf{x})$ is a vector containing the scores for each class for the instance $\mathbf{x}$.

$\sigma(\mathbf{s}(\mathbf{x}))_k$ is the estimated probability that the instance $\mathbf{x}$ belongs to class k , given the scores for each class for that instance.

The function computes the exponential of every score, then normalizes them (dividing by the sum of all the exponentials).

The scores are generally called logits or log-odds (although they are actually unnormalized log-odds).

Softmax Regression

Just like the logistic regression classifier, by default the softmax regression classifier predicts the class with the highest estimated probability (which is simply the class with the highest score), as shown in **Equation 4-21**.

Equation 4-21. Softmax regression classifier prediction

$$\hat{y} = \operatorname{argmax}_k \sigma(\mathbf{s}(\mathbf{x}))_k = \operatorname{argmax}_k s_k(\mathbf{x}) = \operatorname{argmax}_k \left(\left(\boldsymbol{\theta}^{(k)} \right)^\top \mathbf{x} \right)$$

The *argmax* operator returns the value of a variable that maximizes a function. In this equation, it returns the value of k that maximizes the estimated probability $\sigma(\mathbf{s}(\mathbf{x}))_k$.



The softmax regression classifier predicts only one class at a time (i.e., it is multiclass, not multioutput), so it should be used only with mutually exclusive classes, such as different species of plants. You cannot use it to recognize multiple people in one picture.

Training

- Objective: To have a model that estimates a high probability for the target class (and consequently a low probability for the other classes).
- Cost function: Cross Entropy

$$J(\Theta) = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K y_k^{(i)} \log(\hat{p}_k^{(i)})$$

- Minimizing cross entropy cost function should lead to this objective because it penalizes the model when it estimates a low probability for a target class.
- Cross entropy is frequently used to measure how well a set of estimated class probabilities matches the target classes.

Cross Entropy Cost Function

$$J(\Theta) = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K y_k^{(i)} \log \left(\hat{p}_k^{(i)} \right)$$

$y_k^{(i)}$: target probability that the i^{th} instance belongs to class k .

In general, it is either equal to 1 or 0, depending on whether the instance belongs to the class or not.

Notice that when there are just two classes ($K = 2$), this cost function is equivalent to the logistic regression cost function (log loss), which is given by:

Logistic Regression Cost Function:

$$J(\Theta) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log \left(\hat{p}^{(i)} \right) + \left(1 - y^{(i)} \right) \log \left(1 - \hat{p}^{(i)} \right) \right]$$

Cross Entropy

- <https://youtu.be/ErfnhcEV1O8?si=mtc2eFBaOMclf0v8>

Cross Entropy

Entropy

Aurélien Geron, 2018



Entropy:
$$H(p) = -\sum_i p_i \log_2(p_i)$$



Cross Entropy

- **Origin:** Cross entropy originates from Claude Shannon's information theory.
- **Scenario:** Efficiently transmit weather information with 8 options (e.g., sunny, rainy).
- **Encoding Example:**
 - Without assumptions: Use 3 bits per option ($2^3 = 8$).
 - With assumptions: If "sunny" is likely, encode it with 1 bit (e.g., 0), and others with 4 bits.
- **Purpose:** Cross entropy measures the average number of bits sent per option.
- **Perfect Assumption:** Cross entropy equals the weather's entropy (intrinsic unpredictability).
- **Incorrect Assumption:** Cross entropy increases by the Kullback–Leibler (KL) divergence.
- **Formula:** Cross entropy between distributions p and q :

$$H(p, q) = - \sum_x p(x) \log q(x)$$

Why Cross-Entropy

- Alignment with Probability:
 - Softmax outputs probabilities. Cross entropy is designed to compare probability distributions, making it a perfect fit.
- Penalty for Wrong Predictions:
 - Cross entropy heavily penalizes incorrect predictions, especially when the predicted probability for the correct class is low.
- Gradient Properties:
 - The gradient of cross entropy with respect to the logits (before softmax) is straightforward, simplifying the optimization process.
- Optimization Compatibility:
 - Cross entropy loss leads to a convex optimization problem for logistic regression, ensuring efficient and reliable convergence.

Gradient of Cross Entropy Loss

1. Softmax Function

Given an input vector of logits $\mathbf{z} = [z_1, z_2, \dots, z_K]$ where z_k represents the logit (or score) for class k , the softmax function outputs a probability distribution over K classes:

$$\hat{y}_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}$$

where $\hat{y}_k$ is the predicted probability for class k .

2. Cross-Entropy Loss Function

For a single training example, the cross-entropy loss is defined as:

$$L(\mathbf{w}) = - \sum_{k=1}^K y_k \log(\hat{y}_k)$$

where $\mathbf{w}$ represents the model's weights, y_k is the true label (one-hot encoded, so only one $y_k = 1$ and the rest are 0), and $\hat{y}_k$ is the predicted probability for class k .

Gradient of Cross Entropy Loss

We want to compute the gradient of the loss $L(\mathbf{w})$ with respect to the weights $\mathbf{w}_k$.

First, let's express the logits z_k as a linear combination of the input features $\mathbf{x}$ and the weights $\mathbf{w}$:

$$z_k = \mathbf{w}_k^T \mathbf{x} + b_k$$

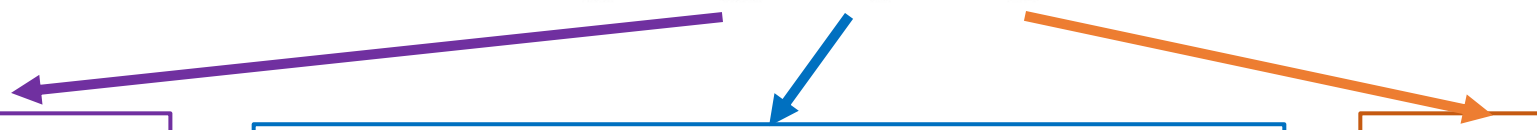
where $\mathbf{w}_k$ is the weight vector for class k and b_k is the bias term.

Now, let's find the gradient of the loss with respect to the weights $\mathbf{w}_k$:

$$\frac{\partial L}{\partial \mathbf{w}_k} = \frac{\partial L}{\partial \hat{y}_k} \cdot \frac{\partial \hat{y}_k}{\partial z_k} \cdot \frac{\partial z_k}{\partial \mathbf{w}_k}$$

We'll calculate each of these terms step by step.

Gradient of Cross Entropy Loss

$$\frac{\partial L}{\partial \mathbf{w}_k} = \frac{\partial L}{\partial \hat{y}_k} \cdot \frac{\partial \hat{y}_k}{\partial z_k} \cdot \frac{\partial z_k}{\partial \mathbf{w}_k}$$


$$\frac{\partial L}{\partial \hat{y}_k} = -\frac{y_k}{\hat{y}_k}$$

For the correct class (when $k = j$):

$$\frac{\partial \hat{y}_k}{\partial z_k} = \hat{y}_k(1 - \hat{y}_k)$$

For the incorrect classes (when $k \neq j$):

$$\frac{\partial \hat{y}_k}{\partial z_j} = -\hat{y}_k \hat{y}_j$$

$$\frac{\partial z_k}{\partial \mathbf{w}_k} = \mathbf{x}$$

1. Partial Derivative of $\hat{y}_k$ with Respect to z_k (Correct Class $k = j$):

The softmax output $\hat{y}_k$ depends on all logits z_j . When taking the derivative of $\hat{y}_k$ with respect to its corresponding logit z_k , we have:

$$\frac{\partial \hat{y}_k}{\partial z_k} = \frac{\partial}{\partial z_k} \left(\frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}} \right)$$

To compute this derivative, we apply the quotient rule (derivative of a quotient) and chain rule:

$$\frac{\partial \hat{y}_k}{\partial z_k} = \frac{e^{z_k} \sum_{j=1}^K e^{z_j} - e^{z_k} \cdot e^{z_k}}{\left(\sum_{j=1}^K e^{z_j} \right)^2}$$

This simplifies to:

$$\frac{\partial \hat{y}_k}{\partial z_k} = \hat{y}_k (1 - \hat{y}_k)$$

Interpretation:

- For the correct class, the derivative reflects how a change in the logit z_k influences the probability $\hat{y}_k$. The term $\hat{y}_k (1 - \hat{y}_k)$ indicates that this influence is larger when $\hat{y}_k$ is close to 0.5 and smaller when $\hat{y}_k$ is close to 0 or 1. This makes sense because probabilities closer to 0 or 1 are more confident predictions, hence less sensitive to small changes in logits.

2. Partial Derivative of $\hat{y}_k$ with Respect to z_j (Incorrect Classes $k \neq j$):

When k and j are not equal, we are taking the derivative of the output probability $\hat{y}_k$ with respect to a different logit z_j :

$$\frac{\partial \hat{y}_k}{\partial z_j} = \frac{\partial}{\partial z_j} \left(\frac{e^{z_k}}{\sum_{l=1}^K e^{z_l}} \right)$$

Applying the quotient rule:

$$\frac{\partial \hat{y}_k}{\partial z_j} = \frac{\left(\sum_{l=1}^K e^{z_l} \right) \cdot 0 - e^{z_k} \cdot e^{z_j}}{\left(\sum_{l=1}^K e^{z_l} \right)^2}$$

This simplifies to:

$$\frac{\partial \hat{y}_k}{\partial z_j} = -\hat{y}_k \hat{y}_j$$

Interpretation:

- For incorrect classes, the derivative represents how the probability $\hat{y}_k$ for class k changes as the logit z_j for another class j changes. The negative sign indicates that increasing z_j decreases the probability $\hat{y}_k$, which makes sense because increasing the logit for an incorrect class should reduce the probability for the correct class in a probabilistic model.

Gradient of Cross Entropy Loss

Gradient for the Correct Class

For the correct class (the one with $y_k = 1$), combining the partial derivatives:

$$\frac{\partial L}{\partial \mathbf{w}_k} = (\hat{y}_k - y_k) \mathbf{x}$$

Since $y_k = 1$ for the correct class:

$$\frac{\partial L}{\partial \mathbf{w}_k} = (\hat{y}_k - 1) \mathbf{x}$$

Gradient for the Incorrect Classes

For incorrect classes (those with $y_j = 0$):

$$\frac{\partial L}{\partial \mathbf{w}_j} = \hat{y}_j \mathbf{x}$$

Gradient of Cross Entropy Loss

Combining the cases for all classes, the gradient of the loss function with respect to the weights $\mathbf{w}_k$ for any class k is:

$$\frac{\partial L}{\partial \mathbf{w}_k} = (\hat{y}_k - y_k) \mathbf{x}$$

This is the gradient used in gradient descent to update the weights:

$$\mathbf{w}_k \leftarrow \mathbf{w}_k - \eta \frac{\partial L}{\partial \mathbf{w}_k}$$

where η is the learning rate.

Gradient Expression

$$\mathbf{w}_k \leftarrow \mathbf{w}_k - \eta \frac{\partial L}{\partial \mathbf{w}_k}$$

Therefore, **all weight vectors** $\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_K$ get updated based on the gradients computed from the same training sample.

The gradient of the cross-entropy loss with respect to the weights in softmax regression is simply the difference between the predicted probability $\hat{y}_k$ and the actual label y_k , scaled by the input features $\mathbf{x}$. This gradient is used in gradient descent to iteratively update the model's weights, reducing the loss and improving the accuracy of the model's predictions.

Cross Entropy → It's Gradient

$$J(\Theta) = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K y_k^{(i)} \log \left(\hat{p}_k^{(i)} \right)$$



$$\nabla_{\Theta}^{(k)} J(\Theta) = \frac{1}{m} \sum_{i=1}^m \left(\hat{p}_k^{(i)} - y_k^{(i)} \right) \mathbf{x}^{(i)}$$

Now you can compute the gradient vector for every class, then use gradient descent (or any other optimization algorithm) to find the parameter matrix Θ that minimizes the cost function.

Coding

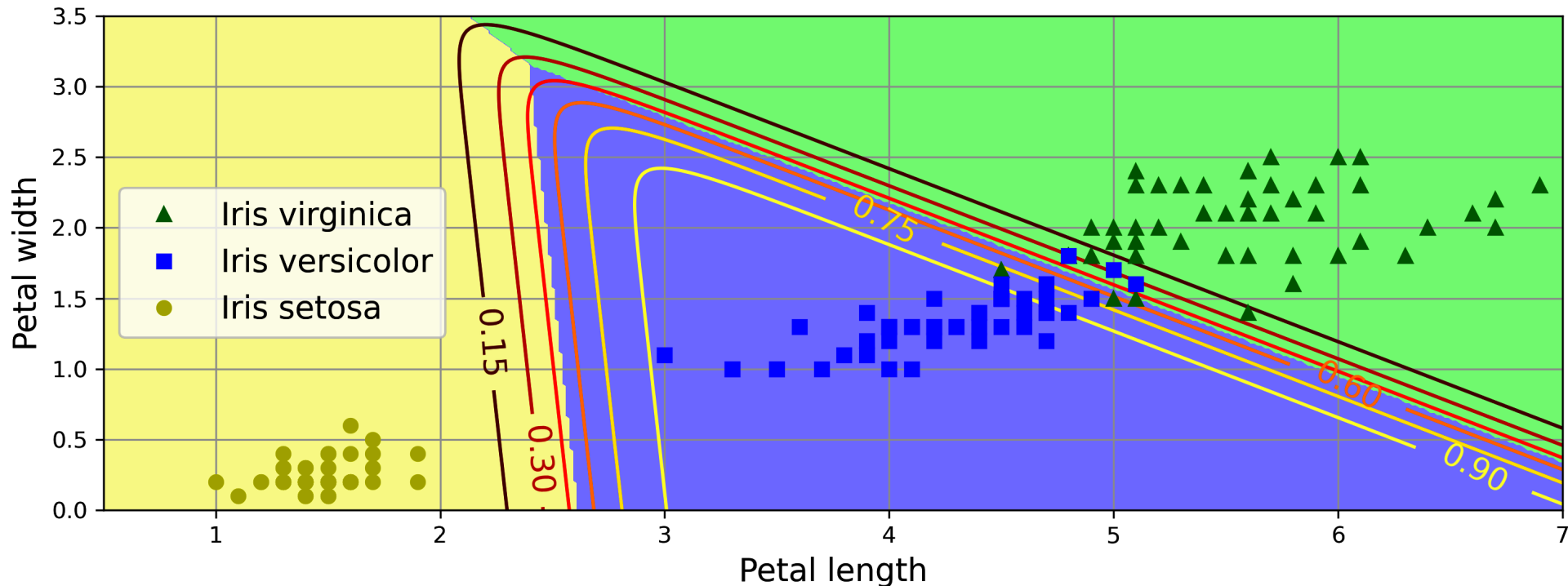
- Scikit-Learn's LogisticRegression:
- Softmax Regression is automatically applied to handle the multi-class classification.
- The "lbfgs" solver is used by default to optimize the cost function, which works well for multi-class problems.
- ℓ_2 Regularization is applied by default to prevent overfitting
- You can control the strength of this regularization using the hyperparameter C.
- smaller C value increases regularization (to avoid overfitting),
- larger C value decreases regularization (allowing the model to fit the training data more closely).

`solver="lbfgs"`

Limited-memory Broyden-Fletcher-Goldfarb-Shanno (L-BFGS or LBFGS) algorithm

- An optimization algorithm
- Particularly well-suited for training models with large numbers of parameters, such as logistic regression or neural networks.
- A quasi-Newton method, meaning it approximates the Newton-Raphson method for finding the minimum of a function
- It does so in a way that is more computationally efficient and requires less memory, especially for high-dimensional problems.

Softmax Regression Decision Boundaries



Notice that the decision boundaries between any two classes are linear. The figure also shows the probabilities for the Iris versicolor class, represented by the curved lines (e.g., the line labeled with 0.30 represents the 30% probability boundary). Notice that the model can predict a class that has an estimated probability below 50%. For example, at the point where all decision boundaries meet, all classes have an equal estimated probability of 33%.



THANK
YOU :)

Acknowledgement

- Slides are based on Textbooks
- Slides found online Many slides including slides by:
 - Saw Shier Nee, Dept. of AI, University of Malaya
 - Andrew Ng, DeepLearning.AI